

Neural Network Compilation: From Foundations to Modern Architectures

Contents

1	Introduction	4
1.1	Neural Networks	4
1.2	Models	4
1.3	Training	4
2	Forward Propagation	5
2.1	Activation Functions	5
2.2	Output and Softmax	6
2.3	Running Example	6
3	Objective Evaluation	7
3.1	Deriving Loss	7
3.2	Regression Loss	8
3.3	Classification Loss	8
3.4	Running Example	9
4	Backpropagation	9
4.1	Learning Rate	9
4.2	Running Example	10
5	Training	11
6	Multiple Neurons	11
6.1	Matrix Formulation	12
6.2	Batching	12
7	Layers and Models	12
7.1	Weight Initialization	13
7.2	Model Structure	13
7.3	Linear Layer	13
7.4	Multilayer Perceptron (MLP)	14
8	Normalization	14
8.1	Normalization Layers	15

9	Convolution	15
9.1	Convolution Layer	16
9.2	Pooling	18
9.3	Convolutional Neural Networks (CNNs)	18
10	Transition to Complex Models	18
10.1	PyTorch	18
10.2	Blocks	19
10.3	Connections	19
11	Large Language Models (LLMs)	19
11.1	Tokenization	20
11.1.1	Running Example	21
11.2	Embedding Layer	21
11.3	Positional Encoding	21
11.4	Attention	22
11.4.1	Multi-Head Attention	23
12	Transformers	23
12.1	Encoder	24
12.2	Decoder	24
12.3	Encoder–Decoder	25
12.4	Running Example	26
13	Training an LLM	27
13.1	Pretraining	27
13.2	Supervised Fine-Tuning (SFT)	27
13.3	Reinforcement Learning from Human Feedback (RLHF)	28
14	Foundation Models	29
15	Diffusion Models	30
15.1	U-Net	30
15.2	Text Encoder	31
15.3	Forward Process	31
15.4	Training Objective	32
15.5	Image Generation	32
A	Appendix	33
A.1	Derivatives	33
A.1.1	Derivative Rules	34
A.1.2	Activation Function Derivatives	35
A.2	Gradients and the Total Differential	35
A.2.1	Total Differential	36
A.3	Probability	36
A.3.1	Properties and Formulas	36
A.3.2	Continuous Variables and the Normal Distribution	37
A.4	Information Theory	37
A.4.1	Common Formulas	37

A.5	Logarithm Properties	38
A.5.1	Properties	38
A.6	Other Identities	38
A.7	Matrix Properties and Calculus	39
A.7.1	Frobenius Inner Product	39
A.7.2	Matrix Calculus	40
A.8	Code Explanation	40
A.8.1	Multi-Layer Perceptron (MLP)	40
A.8.2	Convolutional Neural Network (CNN)	41
A.8.3	Large Language Model (LLM)	41
A.8.4	Diffusion Model	43

1 Introduction

This paper provides an introduction to artificial neural networks, a branch of machine learning that focuses on models inspired by biological neurons. The topics covered range from the fundamental principles of model training to two of the most prominent applications of neural networks: large language models such as ChatGPT and text-to-image models such as DALL-E.

Throughout this paper, certain terms that require background knowledge will appear. These terms will be underlined; clicking on them will navigate to the corresponding section in the appendix, which contains an explanation of the relevant concept.

This paper is accompanied by code in a GitHub repository. The primary topics covered in this paper have accompanying code that readers may run and experiment with.

Link: <https://github.com/GavinMain/NNCompilation>

1.1 Neural Networks

Neural networks are machine learning models inspired by the way humans think. They draw on the principles of pattern recognition, factual understanding, and prediction.

Human neurons function by receiving an input signal, processing it, and transmitting an output to other neurons. Large groups of neurons form specialized circuits—some process visual scenes, others handle auditory noises, others store memories, and some assign emotional value. When a sufficient number of circuits activate in a particular pattern, a thought emerges.

In a similar fashion, the neurons within a neural network are organized into specialized layers. These layers work together to form a model, which is loosely analogous to a human brain.

1.2 Models

In mathematical terms, a model can be thought of as a [function](#) that draws a boundary in the [space occupied by data](#).

In computer science terms, a model can be thought of as the interaction between several layer classes and their forward functions.

The first part of this paper, up to and including matrix representations, focuses more on the mathematical perspective, while the remainder leans more toward the computer science perspective.

1.3 Training

Much like a human learner, neural networks require real-world examples in order to improve. The training process can be broken down into three fundamental steps:

1. Create a thought.
2. Evaluate how far that thought is from the ideal answer.
3. Adjust thinking patterns based on the evaluation.

Consider the saying “a burned hand teaches best.” In the first step, a person decides to touch a hot surface. In the second step, the hot surface causes pain. In the third step, the person learns to avoid touching hot surfaces in the future. In neural network terminology, these three steps correspond to forward propagation, objective evaluation, and backpropagation.

2 Forward Propagation

Every person has a set of preferences that guide their decisions. Someone who enjoys candy might favor sweet flavors while disliking bitter ones. A neural network assigns each of these preferences a numerical value called a weight. For example:

$$w_{\text{sweet}} = 5, \quad w_{\text{bitter}} = -4.$$

When a person encounters something new—say, coffee with sugar—they compare the characteristics of the coffee against their preferences to decide whether they would like it. A neural network receives its input as numbers, so the coffee might be represented as:

$$x_{\text{sweet}} = 2, \quad x_{\text{bitter}} = 3.$$

A simple [linear](#) model can multiply the weights by the input to produce an output:

$$h = w_{\text{sweet}} \cdot x_{\text{sweet}} + w_{\text{bitter}} \cdot x_{\text{bitter}}.$$

There are several problems with this model. For one, most data does not be properly separated using a single linear equation. This could partially be remedied by adding more layers (e.g., $h_2 = Wh_1$), but stacking linear layers together still produces a linear model with no complex separation.

$$y = W_1x + b_1, \tag{1}$$

$$y = W_3(W_2(W_1x + b_1) + b_2) + b_3, \tag{2}$$

$$y = W_3W_2W_1x + W_3W_2b_1 + W_3b_2 + b_3, \tag{3}$$

$$y = W'x + b'. \tag{4}$$

Here, after combining three linear layers, the output reduces to a single linear layer; albeit with a different weight and bias. (The bias is a constant term that allows the model to shift the output vertically, providing more flexibility in fitting data that does not pass through the origin. Modern neural networks sometimes omit the bias because other components of the architecture can compensate for its effects.)

2.1 Activation Functions

The concept of a nonlinear activation function addresses this limitation. By applying an activation function $z = \sigma(h)$, the model transforms from a straight line into a complex curve.

The following are some common activation functions:

- Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x)$$

- GELU (Gaussian Error Linear Unit):

$$\text{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \sigma(1.702 x)$$

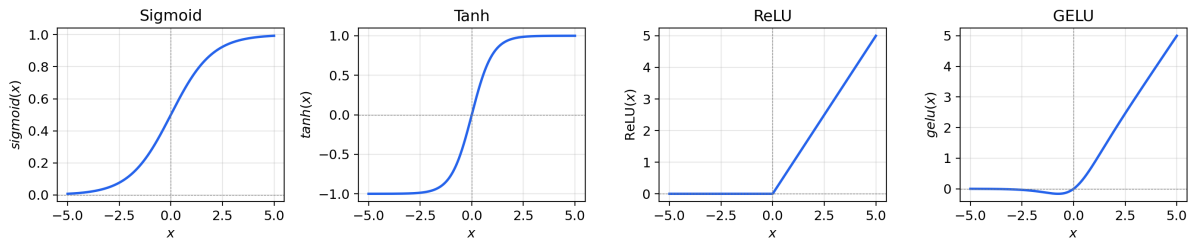


Figure 1: Common activation functions and their shapes.

2.2 Output and Softmax

By the end of forward propagation, the model outputs one or more number values. In regression tasks, the output can be used directly, or be passed into sigmoid or tanh to constrain it to a specific range. Sigmoid is often used when the output represents a probability (0 to 1), while tanh is used when the output represents a score (-1 to 1).

In classification tasks, the output is typically an array of logits - a logit is a real number of the form $\log(p)$. Models do not explicitly output logits - in that they are not trained to output a value in a specific form or equation - but the training process treats model outputs as logits due to their convenient properties. Logits are often passed through the [softmax function](#):

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}},$$

to transform them into a proper probability distribution where each value is a probability between 0 and 1, and all values sum to 1. (Logits are convenient because $e^{\log(p)} = p$).

2.3 Running Example

Returning to the example, the model consists of a single linear layer for simplicity:

$$\text{out} = w_{\text{sweet}} \cdot x_{\text{sweet}} + w_{\text{bitter}} \cdot x_{\text{bitter}} = (5)(2) + (-4)(3) = 10 - 12 = -2.$$

There are many ways to interpret this result. This example will be treated as a regression problem, where 0 indicates no opinion and increasingly positive or negative values indicate how strongly the model favors or disfavors the coffee.

3 Objective Evaluation

Suppose the hypothetical person decides not to drink the coffee because their opinion is numerically equivalent to -2 , but a friend later recommends it, saying it should be a 5 . After trying the coffee, the person discovers that they actually enjoy it and reflects on their thought process.

In neural networks, the objective function evaluates the performance of the model by comparing its output against the true labels. The true labels are the ideal answers that the model should have predicted. The three main learning paradigms differ in how these labels are provided:

- **Supervised learning:** The true labels are given to the model by the programmer.
- **Unsupervised learning:** The true labels are not provided, and the model must learn to identify patterns without explicit guidance. Models have to get creative when trying to learn without labeled data and often the goal is not the final output, but an intermediary step in the process.
- **Reinforcement learning:** The true labels are replaced by rewards, which are derived from the model's actions in an environment. The model learns to maximize its rewards over time.

This paper mostly covers supervised learning, but the other learning paradigms may be mentioned in some sections.

3.1 Deriving Loss

In supervised learning, the maximum likelihood estimation (MLE), formulates the idea that the best parameters $\hat{\theta}$ are those that maximize the probability of predicting the data given the model's weights $p(\text{data} \mid \theta)$:

$$\hat{\theta} = \arg \max_{\theta} p(\text{data} \mid \theta).$$

The data consists of input-output pairs (x, y) . The model should predict each pair correctly: given input x , the model should output y . Since these predictions are [independent](#), their probabilities multiply

$$\hat{\theta} = \arg \max_{\theta} \prod_{(x,y)} p(y \mid x, \theta).$$

Multiplying many small probabilities leads to numerical underflow (values approaching zero), making this product difficult to optimize. Taking the [logarithm](#) of both sides resolves this issue, since the product rule converts the product into a sum. This gives the log-likelihood:

$$\hat{\theta} = \arg \max_{\theta} \sum_{(x,y)} \log p(y \mid x, \theta).$$

(The log of the parameters still gives model parameters, so the $\arg \max$ is unaffected by the log transformation.)

In supervised learning, the model wants to minimize the difference between its output and the true label, but the current formula maximizes instead. This can be resolved using the [negation identity](#), which reformulates the problem as minimizing the negative log-likelihood:

$$\mathcal{L} = - \sum_{(x,y)} \log p(y \mid x, \theta).$$

$\mathcal{L}$ is the loss function. Different models define $p(y \mid x, \theta)$ differently, leading to different specific loss functions.

3.2 Regression Loss

In regression, the model predicts a real number. For instance, given humidity and time of day, a model might predict the temperature.

This can be framed as follows: the target output y equals the predicted output plus some error, where the error follows a [normal distribution](#). That is, $y \sim \mathcal{N}(f_\theta(x), \sigma^2)$, where $f_\theta(x)$ is the model's prediction.

Starting from the negative log-likelihood and substituting the Gaussian PDF:

$$\hat{\theta} = \arg \min_{\theta} \sum_i -\log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(y_i - f_\theta(x_i))^2}{2\sigma^2} \right) \right] \quad (5)$$

$$= \arg \min_{\theta} \sum_i \left[-\log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) + \frac{(y_i - f_\theta(x_i))^2}{2\sigma^2} \right]. \quad (6)$$

In machine learning, constants do not affect the optimization of the parameters. All constants can be dropped to obtain the sum of squared errors (SSE):

$$\text{SSE} = \sum_{i=1}^N (y_i - f_\theta(x_i))^2.$$

The problem with SSE is that it is not very stable—as the number of (x, y) pairs increases, so does the loss. To address this, the average is taken, resulting in the mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - f_\theta(x_i))^2.$$

When implementing MSE, programmers often multiply this formula by $\frac{1}{2}$ to simplify the [power rule](#) computation during differentiation:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2} (y - \hat{y})^2 \quad (\text{single data point}).$$

3.3 Classification Loss

In classification, the model outputs a list of logits that are then passed through softmax to become probabilities. The probability of correctly predicting the target is $p(\text{class}_i)$. For example, a model that predicts whether an image contains a cat or a dog might

produce the post-softmax output $[0.2, 0.8]$. If the data defines element 0 as “cat” and element 1 as “dog,” then $p(\text{dog}) = 0.8$.

Starting from the general loss formula:

$$\mathcal{L} = - \sum_i \log p(\text{class}_i).$$

This is equivalent to the [cross-entropy](#) formula from information theory, summed over all data points. For this reason, it is called cross-entropy loss.

Implementations of cross-entropy loss often combine the softmax function and the loss computation into a single operation because models are expected to output logits rather than probabilities.

3.4 Running Example

The hypothetical person predicted a score of -2 for the coffee, but after tasting it, determined the score should be 5. Since example is treated as a regression problem, the mean squared error loss is used:

$$\mathcal{L} = \frac{1}{2}(y - \text{out})^2 = \frac{1}{2}(5 - (-2))^2 = \frac{1}{2}(7)^2 = 24.5.$$

The person reflects that they need to adjust their preferences by an amount proportional to this loss.

4 Backpropagation

In backpropagation, every operation from the forward pass and loss computation is differentiated in reverse order (from output back to input). Weights and biases are updated using:

$$W \leftarrow W - \frac{\partial \mathcal{L}}{\partial W}, \quad \mathbf{b} \leftarrow \mathbf{b} - \frac{\partial \mathcal{L}}{\partial \mathbf{b}}.$$

In multilayer networks, each layer contributes a different amount to the final loss. To account for this, each layer passes $\frac{\partial \mathcal{L}}{\partial X}$ (the gradient with respect to its input) backward to the previous layer. The final layer receives $\frac{\partial \mathcal{L}}{\partial \text{out}}$ because its output is the input to the loss function.

4.1 Learning Rate

The objective of supervised learning is to minimize the loss. In the objective evaluation step, the loss is calculated and in the backpropagation step, the model’s weights are updated to reach the minimum.

This is done by moving the weights one step at a time towards the minimum using the [gradient](#) of the objective function. When stepping towards the minimum, if each update step is too large, the model can overshoot the minimum entirely, landing on the other side of the “valley”:

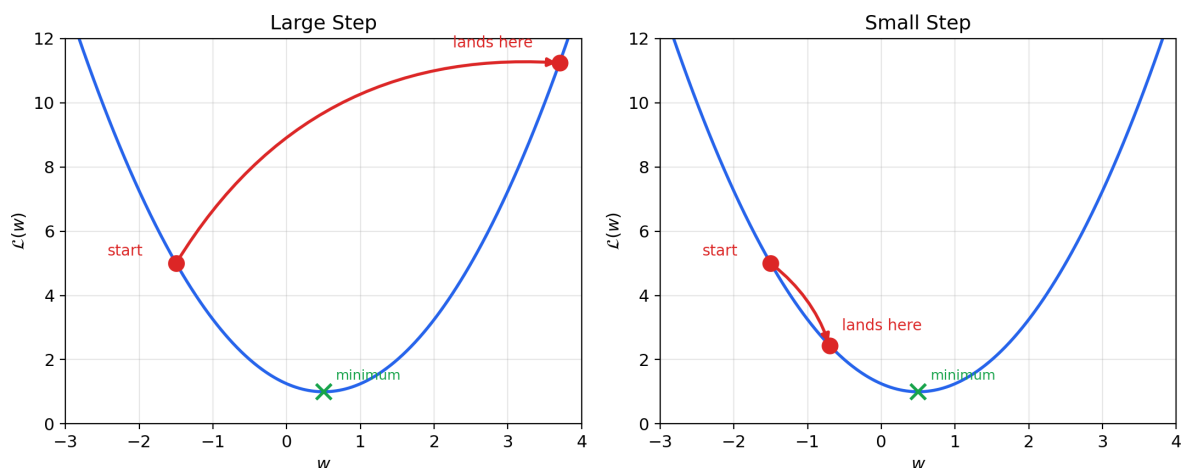


Figure 2: A large learning rate can overshoot the minimum, while a small learning rate converges steadily.

This issue is addressed by introducing a learning rate α . Instead of updating the weight by the full gradient, only a small fraction is applied. This works because a gradient has both a magnitude and a direction. The learning rate reduces the magnitude without altering the direction. More steps will be needed but each step is stable and is less likely to overshoot the minimum. The weight update formula becomes:

$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}}{\partial W}.$$

4.2 Running Example

After reflecting, the person understands why they enjoyed the coffee and needs to update their preferences accordingly. Using the values from the running example ($x_{\text{sweet}} = 2$, $x_{\text{bitter}} = 3$, target $y = 5$, prediction out = -2, learning rate $\alpha = 0.1$):

$$\mathcal{L} = \frac{1}{2}(y - \text{out})^2, \quad \text{out} = w_{\text{sweet}} \cdot x_{\text{sweet}} + w_{\text{bitter}} \cdot x_{\text{bitter}}.$$

Applying the [chain rule](#):

$$\frac{\partial \mathcal{L}}{\partial w_{\text{sweet}}} = \frac{\partial \mathcal{L}}{\partial \text{out}} \cdot \frac{\partial \text{out}}{\partial w_{\text{sweet}}}.$$

Computing each part (using the power rule and the chain rule):

$$\frac{\partial \mathcal{L}}{\partial \text{out}} = -(y - \text{out}) = -2 - 5 = -7, \quad \frac{\partial \text{out}}{\partial w_{\text{sweet}}} = x_{\text{sweet}} = 2.$$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial w_{\text{sweet}}} = (-7)(2) = -14.$$

The weight update:

$$w_{\text{sweet}} \leftarrow 5 - (0.1)(-14) = 5 + 1.4 = 6.4.$$

The same procedure is repeated for w_{bitter} :

$$\frac{\partial \mathcal{L}}{\partial w_{\text{bitter}}} = (-7)(3) = -21, \quad w_{\text{bitter}} \leftarrow -4 - (0.1)(-21) = -4 + 2.1 = -1.9.$$

5 Training

So far, the running example has completed only one epoch. An epoch is defined as one full pass of training (forward propagation, loss computation, and backpropagation) over all data pairs. In this case, there is only one data pair. The following figure illustrates the effect of training the example model over several epochs. The horizontal axes represent x_{sweet} and x_{bitter} respectively. The vertical axis represents the model's output. The goal is for the model to predict 5 (indicated by the red dot) when the input is:

$$x_{\text{sweet}} = 2, \quad x_{\text{bitter}} = 3.$$

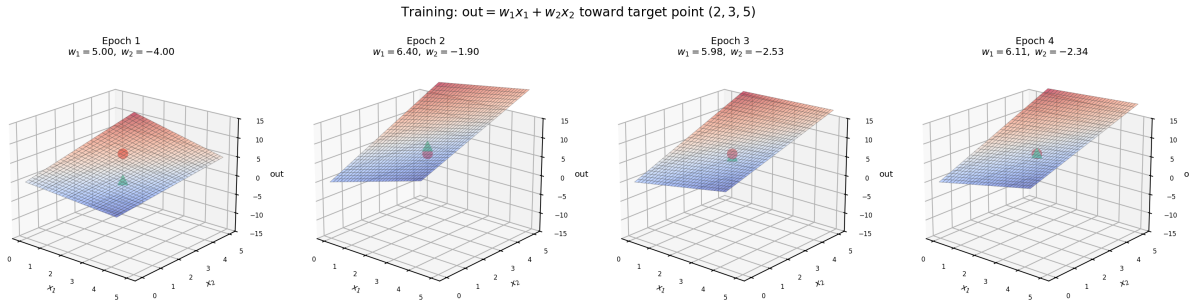


Figure 3: The surface $\text{out} = w_1x_1 + w_2x_2$ across four epochs of training. The red sphere marks the target (2, 3, 5); the green triangle marks the model's prediction. With each epoch, triangle gets closer to the sphere.

6 Multiple Neurons

So far, only a single neuron has been considered. Data consists of points in some space—two-dimensional, three-dimensional, or higher. A single neuron can create only one boundary (a line or curve) to separate data. Adding more neurons creates additional boundaries, carving the space into more complex regions.

With one neuron:

$$\text{out}_1 = w_1 \cdot x_1 + w_2 \cdot x_2 + \text{bias}_1.$$

Adding a second neuron:

$$\text{out}_2 = w_3 \cdot x_1 + w_4 \cdot x_2 + \text{bias}_2.$$

To manage this efficiently (rather than creating individual variables for each neuron), [matrices](#) are used.

6.1 Matrix Formulation

The input $\mathbf{x}$ is now a row vector $[x_{\text{sweet}}, x_{\text{bitter}}]$. For one neuron, the weight vector is a single column. In matrix notation:

$$W = \begin{bmatrix} w_{11} \\ w_{21} \end{bmatrix}.$$

To add a second neuron, an additional column is appended (each column represents one neuron):

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}.$$

The bias is a $1 \times \text{neurons}$ vector (one bias per neuron). The forward formula becomes:

$$\text{out} = Y = XW + \mathbf{b}, \quad \mathbf{b} = [b_1 \quad b_2].$$

For backpropagation, $\frac{\partial \mathcal{L}}{\partial W}$ and $\frac{\partial \mathcal{L}}{\partial X}$ are needed. Using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial Y} \cdot \frac{\partial Y}{\partial W}, \quad \frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} \cdot \frac{\partial Y}{\partial X}.$$

Normally, for linear layers, $\frac{\partial Y}{\partial W} = X$ and $\frac{\partial Y}{\partial X} = W$. However, the shapes do not align for matrix multiplication. For example, assume X is $m \times k$ and W is $k \times n$. $\frac{\partial \mathcal{L}}{\partial Y}$ would be $m \times n$, but X is $m \times k$.

To resolve this, the transpose of X is used. This comes from the [total differential](#):

$$dF = \left\langle \frac{\partial F}{\partial X}, dX \right\rangle$$

This is a [Frobenius inner product](#), which can be expressed as a matrix multiplication of the transpose of $\frac{\partial F}{\partial X}$ and dX . Therefore, $\frac{\partial \mathcal{L}}{\partial W} = X^T \cdot \frac{\partial \mathcal{L}}{\partial Y}$ and $\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} \cdot W^T$.

6.2 Batching

So far, only a single input ($1 \times \text{features}$) is being processed at a time. This is inefficient due to per-sample overhead. Instead, multiple data points can be processed simultaneously as a batch ($\text{batch} \times \text{features}$). The weight matrix ($\text{features} \times \text{neurons}$) and bias ($1 \times \text{neurons}$) remain unchanged because the nature of matrix multiplication already makes every row in the input get multiplied by every neuron. The output shape becomes $\text{batch} \times \text{neurons}$, and a corresponding batch of targets is used to compute the loss.

7 Layers and Models

With the theory out of the way, the next step is to create models in code. The basic building block of a model is a single layer consisting of weights, forward, and backward functions. The forward function produces an output like $\text{out} = XW + \mathbf{b}$, while the backward function computes the gradients $\frac{\partial \mathcal{L}}{\partial W}$ and $\frac{\partial \mathcal{L}}{\partial X}$ and updates the weights.

7.1 Weight Initialization

When a layer is created, its weights must be initialized. The initialization method depends on the activation function. Two of the most common methods are:

- Xavier initialization [2] (used with Sigmoid and Tanh):

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right),$$

where n_{in} and n_{out} are the number of input and output features, respectively.

- He initialization [3] (used with ReLU and GELU):

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right).$$

7.2 Model Structure

A model is a class that consists of multiple layers along with a forward and backward function. The forward function calls each layer's forward function sequentially and applying activation functions between layers:

$$\mathbf{x} \leftarrow \text{layer}_1.\text{forward}(\mathbf{x}), \quad (7)$$

$$\mathbf{x} \leftarrow \text{activation}(\mathbf{x}), \quad (8)$$

$$\mathbf{x} \leftarrow \text{layer}_2.\text{forward}(\mathbf{x}). \quad (9)$$

The backward function traverses the layers in reverse order, passing the input gradient ($\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$) through each layer:

$$\text{gradient} \leftarrow \text{layer}_2.\text{backward}(\text{gradient}), \quad (10)$$

$$\text{gradient} \leftarrow \text{activation_backward}(\text{gradient}), \quad (11)$$

$$\text{gradient} \leftarrow \text{layer}_1.\text{backward}(\text{gradient}). \quad (12)$$

7.3 Linear Layer

There are many types of layers, each defined by its weights and forward function. The linear layer is the most fundamental and versatile layer. Its weights are 2D matrices where each column represents a neuron or boundary. Its forward formula is:

$$\mathbf{z} = XW + \mathbf{b}.$$

The linear layer's primary role is to change the shape of the data, converting an input of shape (batch, in_features) to an output of shape (batch, out_features). This functionality is useful for defining output classes or reshaping data for subsequent parts of a model.

7.4 Multilayer Perceptron (MLP)

The multilayer perceptron (MLP) is the most general-purpose neural network. It makes no assumptions about the structure of its input data and, in principle, has the ability to approximate any continuous function (a property known as the universal approximation theorem). This generality means that the MLP performs reasonably well on a wide range of tasks but is not necessarily optimal for any specific one.

An MLP consists of an input layer, at least one hidden layer, and an output layer:

$$\text{input layer: } \text{LinearLayer}(\text{input_size}, \text{hidden_size}), \quad (13)$$

$$\text{hidden layer: } \text{LinearLayer}(\text{hidden_size}, \text{hidden_size}), \quad (14)$$

$$\text{output layer: } \text{LinearLayer}(\text{hidden_size}, \text{output_size}). \quad (15)$$

8 Normalization

The learning rate addresses the magnitude of the weight update but does not fix the gradient itself. For example, the derivative of the sigmoid function:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)),$$

has a range of $[0, 0.25]$. In a deep model with many sigmoid activations, the gradient shrinks with each layer until it eventually becomes too small to meaningfully affect the weights. This issue is called the vanishing gradient. Conversely, if $\frac{\partial \mathcal{L}}{\partial X}$ consistently has a magnitude greater than 1, the gradient can grow uncontrollably—a problem known as the exploding gradient.

The most widely applicable method for addressing these issues is normalization.

Note: In statistics, this process is known as *standardization*; the term “normalization” has a different meaning in that field. This section uses “normalization” in the computer science sense.

Data often arrives in a variety of ranges. For example, temperature is usually in the range 32–100 °F, while the price of an ice cream cone is in the range \$1–\$3. If a model were to predict whether someone would buy an ice cream cone, the 100 °F would carry much more weight than the \$3 price tag. In machine learning, every feature should start with the same contribution potential and only get phased out during the learning process. Temperature having an initial advantage due to its higher range runs contrary to this principle.

Normalization addresses this issue. The general formula is:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}},$$

where μ is the mean, σ^2 is the variance, and ϵ is a small constant to prevent division by zero. After normalization, the data has a mean of 0 and a variance of 1.

This addresses the vanishing and exploding gradient problems by constraining the data and gradients to a predictable range.

8.1 Normalization Layers

The above formula is used during data preprocessing. However, each layer in a neural network may shift the mean and variance of its outputs, destroying the range from the preprocessing step. To maintain stability throughout the entire model, normalization layers are introduced within the network.

Like other layers, normalization layers have weights and forward functions. The forward function first normalizes the data:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}},$$

and then applies a learned scale (γ) and shift (β) (analogous to weights):

$$\text{output} = \gamma \hat{x} + \beta.$$

The result is that the output has mean β and variance γ^2 . The scale and shift is to ensure that the distribution is predictable, but not rigidly fixed to mean 0 and variance 1.

Different types of normalization layers exist, differing in how μ and σ^2 are computed. Consider an input with shape (batch, C , H , W), representing batch size, channels, height, and width (this is the standard representation for images):

- Layer normalization computes the mean and variance over a single data point (across C , H , and W).
- Batch normalization computes the mean and variance over the entire batch for each channel. That is, it normalizes across (batch, H , W) and repeats this for each channel independently.
- Group normalization divides the channels into groups and computes the mean and variance for each group across (C_{group} , H , W).

9 Convolution

Convolutional neural networks are models that excel at spatial feature extraction. It primarily functions using the convolution operation. The following figure shows an input image on the left and the resulting image after applying a kernel (sometimes called a filter) on the right.

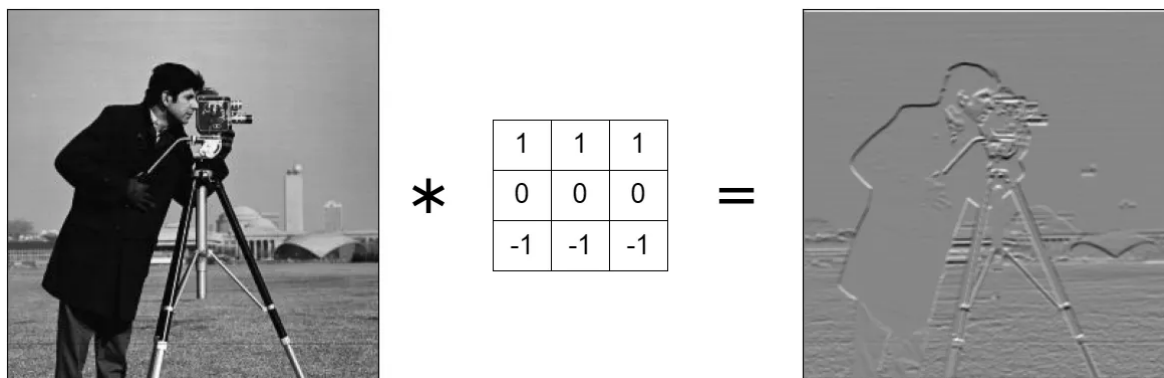


Figure 4: Applying a kernel to an image.

A kernel is a small matrix. Researchers discovered that kernels with specific values can perform specific operations. For instance, the kernel in the example above detects horizontal edges. The images below are a result of applying different kernels to an original image:

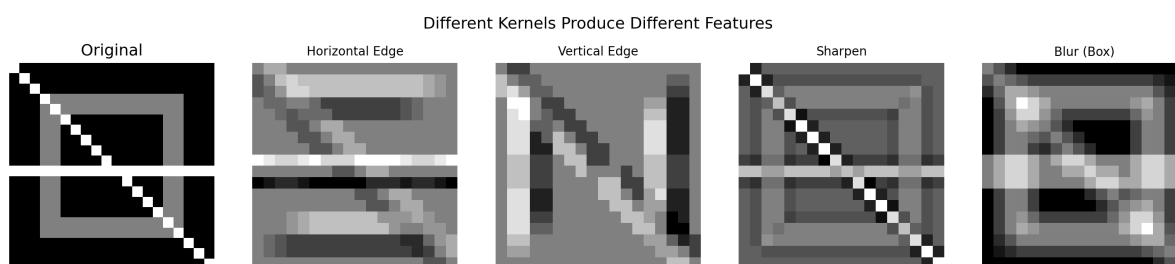


Figure 5: Different kernels detect different features in an image.

The convolution operation applies a kernel to an image as follows:

1. Place the kernel at one corner of the image.
2. Compute the element-wise product between the kernel and the corresponding patch of the image, then sum the results to produce a single value in the output.
3. Move the kernel by a fixed number of positions (called the stride) and repeat.

The output size for a given dimension is:

$$\text{output_size} = \frac{\text{input_size} - \text{kernel_size} + 2 \times \text{padding}}{\text{stride}} + 1.$$

The output image is generally smaller than the input. To address this, the concept of padding is used, which involves surrounding the image with zeros, effectively enlarging it before the convolution is applied.

9.1 Convolution Layer

In the examples above, kernels had fixed values designed by researchers. In a convolution layer, the kernel values are learned during training—that is, the kernel values serve as the trainable weights.

Images are typically represented in the form (channels, height, width). The height and width are spatial dimensions. The channels represent features: in the first layer, there may be 3 channels corresponding to the red, green, and blue (RGB) color components. Even grayscale images have one channel ($1 \times H \times W$). In later layers, channels may represent higher-level features such as edges, curves, or shapes.

A kernel in a convolution layer is three-dimensional: ($\text{in_channels} \times \text{kernel_size} \times \text{kernel_size}$). During the convolution operation, the kernel examines a patch across all input channels, computes a dot product per channel, and sums the values of every channel to produce a single scalar. One kernel produces one output channel. A convolution layer contains multiple kernels, each producing its own output channel.

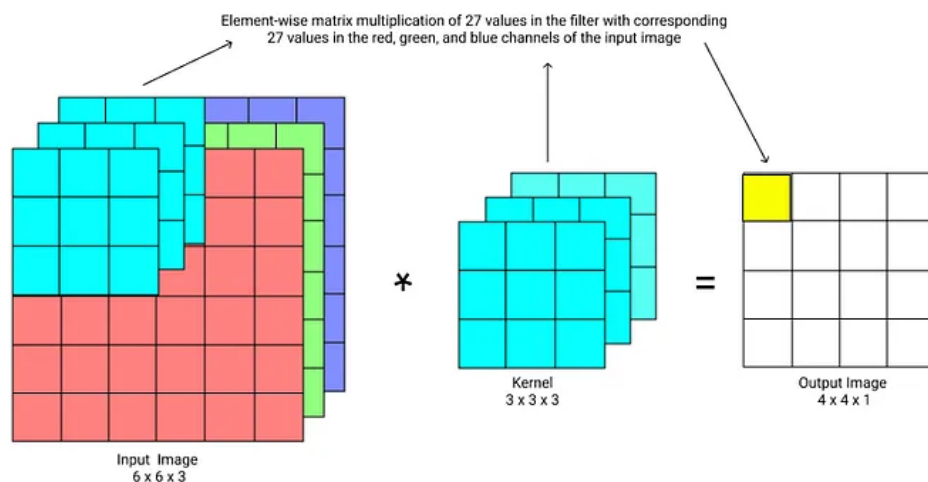


Figure 6: A single three-dimensional kernel spans all input channels and produces one output channel.

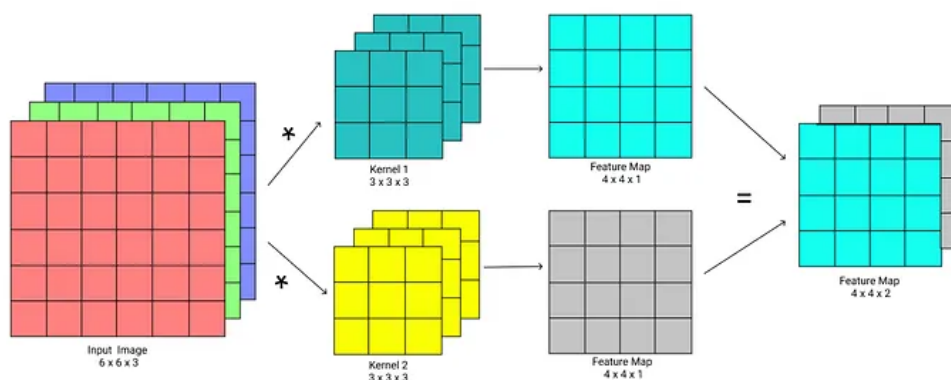


Figure 7: Multiple kernels each produce one output channel; stacking them yields a multi-channel output.

Unlike the linear layer (which specifies input and output sizes), the convolution layer is defined by the number of input channels, output channels, and kernel size. The exact shape of the input is not needed because convolution applies to any sized images. Padding and stride can also be configured to change output size per channel. The forward function applies the learnable kernel (weights) to the input.

9.2 Pooling

A convolution layer’s output size depends on the kernel size, padding, and stride. While these parameters can be varied to change the output dimensions, doing so may sacrifice spatial detail. A separate pooling operation can be used instead. Pooling resembles applying a kernel, but instead of computing a dot product, it applies an aggregation function to each patch. For example, max pooling takes the maximum value from each patch. The same output size formula from the convolution section applies to pooling.

9.3 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks are specialized for spatial data represented on grids. Most commonly, this means image processing, but CNNs can also be applied to audio (since audio can be converted into spectrograms) and also board games. A CNN typically consists of several stages of convolution, pooling, and activation, followed by a linear layer at the end to produce the desired output size.

10 Transition to Complex Models

The previous sections assume that the models are being implemented on a CPU. In practice, MLPs are the only models that can be trained efficiently on a CPU. Training a CNN on a CPU is extremely slow. The primary reason is that matrix multiplication can be decomposed into many smaller, independent operations that run in parallel. CPU cores are powerful and handle branching logic well, but for large volumes of simple arithmetic, quantity of cores matters more. Consumer-grade CPUs typically have 8–32 complex cores, whereas GPUs have tens of thousands of simple cores that can act in parallel. The problem is that implementing a CNN on a GPU requires an entirely different codebase. This section introduces the tools and concepts needed to transition from CPU-based models to GPU-based models, while also abstracting away the tedious aspects of model creation.

10.1 PyTorch

PyTorch is one of the most widely used Python libraries for neural networks. Beyond providing well-optimized code, it offers two primary benefits:

1. PyTorch can run models on GPUs, which drastically increases training speed.
2. PyTorch automatically computes gradients. Rather than manually computing $\frac{\partial \mathcal{L}}{\partial out}$ and passing it to a backward function, PyTorch records every forward operation in a computation graph. Once the loss is computed (e.g., `loss = cross_entropy(...)`), calling `loss.backward()` automatically computes the gradients for all operations using the internal computation graph.

Rather than performing the simple update $w \leftarrow w - \alpha \cdot \frac{\partial \mathcal{L}}{\partial w}$, PyTorch provides optimizers such as Adam, that incorporates features like adaptive learning rates and momentum. Calling `optimizer.step()` applies the gradient updates to all parameters. This allows model implementations to bypass the need for a backward function effectively saving 33% of the code required.

10.2 Blocks

As neural network architectures grow in complexity, more and more layers are added, and the forward function becomes increasingly cluttered. To manage this complexity, the concept of a block is introduced. A block is just another name for a model, but it is often used as a building block for more complex models. There are two main reasons to use blocks:

First, if a particular architecture is repeated multiple times within a model, it can be encapsulated as a block to simplify the forward function. For example, ResNet18, a complex Convolutional Neural Network, consists of 8 convolution + activation + normalization + pooling layers. Instead of writing out all 32 layers in the forward function, a ResBlock is defined as two convolution + activation + normalization layers, and the forward function simply calls the ResBlock 4 times.

Second, if a particular architecture serves a well-defined purpose, it may be considered a block even if its internal structure varies across implementations. The most well-known example is the encoder–decoder architecture, which consists of an encoder block and a decoder block. An encoder block converts input data into a different, typically compressed, representation. A decoder block takes that representation and reconstructs the original input data. Encoder–decoder architectures can be implemented using various layers, including MLPs and CNNs. Some well known examples include variational autoencoders, U-Nets, and Transformers.

10.3 Connections

In complex models, earlier layers’ contributions may be lost as the data passes through many layers and transformations. In addition, gradients may vanish or explode during backpropagation. To address this, connections are introduced. There are two well-known types of connections:

- Residual connections: These are usually present in deep CNNs, which have large blocks with many layers. A residual connection adds the input of a block to its output, allowing the original input to bypass the block’s transformations, while also creating a shortcut for gradients to flow through.
- Skip connections: These are usually present in encoder–decoder architectures. A skip connection concatenates the output of a layer in the encoder directly to the input of a layer in the decoder. This allows the decoder to access information from much earlier in the model while also providing a shortcut for gradients to flow through. Skip connections are especially important in encoder–decoder architectures because the encoder compresses the data, which can lead to loss of information. The skip connection allows the decoder to access the original, uncompressed data, improving the model’s ability to reconstruct the input.

11 Large Language Models (LLMs)

Arguably the most well-known application of deep learning are large language models such as ChatGPT. At their core, LLMs are advanced classification models: given a partial sequence of text, they output a probability distribution over all possible next tokens. In addition to the standard forward and backward functions, LLMs have a generation

function. This function runs the forward pass to predict one token, appends that token to the current sequence, and feeds the extended sequence back into the forward pass. This process repeats until a predefined token limit is reached or some other stopping condition is met.

This section goes over the architecture of LLMs, starting with the input data and ending with the output probabilities. The next section goes over the training process, which consists of several phases with different objectives and data structures.

11.1 Tokenization

Consider the following example sentence:

The dog ran over to the ball and it was round.

All data passed into a model must be numerical (for matrix multiplication). The tokenization step takes unique substrings, called tokens, and maps each one to a unique number called a token ID.

For the word “dog,” possible tokens might include: “dog,” “d, o, g” “do, g” or “d, og.”

A tokenizer implementation requires three main functions:

1. Build: Creates a vocabulary by defining what constitutes a token and mapping each token to a unique ID.
2. Encode: Translates a sequence of tokens (text) into a sequence of IDs.
3. Decode: Translates a sequence of IDs back into tokens.

There are various ways to tokenize text. The simplest approach treats each character or byte as a single token. This is simple and universal, but the small vocabulary size greatly increases the sequence length. For example, “dog” would become three separate tokens: [d:1, o:2, g:3].

Byte pair encoding is a method that begins by treating each byte as a token, then iteratively merges the most frequently co-occurring pairs. This merging process is deterministic and greedy: it always selects the pair with the highest frequency. Without a stopping condition, the process could continue until the entire text is a single token. Common stopping conditions include reaching a target vocabulary size or only merging pairs that exceed a minimum frequency threshold.

After BPE, the word “dog” might have the token entries [d:1, do:2, dog:3]. During encoding, the tokenizer selects the longest matching token, so “dog” maps to a single ID. The GPT family of models uses this type of tokenizer, as does the accompanying code.

Note that tokenizers are tied to their models. Models are trained on specific token IDs, so if the IDs map to different tokens, the model will produce meaningless output.

Aside from the vocabulary generated by the build function, tokenizers must reserve space for special tokens – unique substrings that do not appear naturally in the training text, which are used to tell the model specific information. For example, if the LLM should stop generating before reaching a token limit (e.g., after completing a sentence), it must be trained to produce an end-of-text token and ending its generation loop once it detects the token. The end-of-text token is one of the most widely used special tokens, along with the padding and unknown tokens.

11.1.1 Running Example

Using one token per word tokenization on the example sentence produces:

The dog ran over to the ball and it was round.
[1, 2, 3, 4, 5, 1, 6, 7, 8, 9, 10]

11.2 Embedding Layer

After tokenization, each token is represented by a single number. However, a single number carries very limited information. For comparison, the [MNIST](#) images use 784 numbers to represent one image, and even the simplest intuition example in this paper used two numbers (sweet and bitter).

The embedding layer transforms each token ID into an embedding vector—a list of numbers that represents a point in a high-dimensional space. The idea behind embeddings is that tokens with similar meanings should be close to each other in this space, while tokens with different meanings should be far apart. In a two-dimensional space, there are only two axes available for distinguishing words. For example, one axis might represent formal definitions and the other might capture social connotations. The word “great” formally carries a positive meaning, but in a sarcastic context, it can convey something negative.

Consider the words “medium” and “mid.” Along the formal axis, both represent the midpoint. However, “medium” carries little social connotation, while “mid” is often used informally to mean “below average.” Two dimensions do not fully capture the meaning of these words—for instance, “medium” can refer to steak doneness, which relates to the word “rare” (as in “medium rare”), yet “rare” (meaning uncommon or above average) is very different from “medium” (meaning average). The more dimensions available, the more nuances of meaning can be captured. The trade-off is that higher dimensions lead to larger models, and beyond a certain point, there are diminishing returns.

The embedding layer is implemented as a matrix of shape (vocab_size × embedding_dim). Each row corresponds to one token in the vocabulary, and each column represents one dimension. During the forward pass, a token ID is used as an index to retrieve the corresponding row. During backpropagation, only the retrieved row receives a gradient update; all other rows have zero gradient.

In practice, a sequence of IDs is passed in at once, and the embedding layer returns all corresponding rows. The output is a (sequence_length × embedding_dim) matrix.

Continuing with the example, the token IDs [1, 2, ..., 10] are passed into the embedding layer, which outputs:

$$\begin{bmatrix} \text{— embedding vector for token 1 —} \\ \text{— embedding vector for token 2 —} \\ \vdots \\ \text{— embedding vector for token 10 —} \end{bmatrix}.$$

11.3 Positional Encoding

The embedding vectors carry no information about the order of tokens in the sequence. This is not ideal as the order of words can change the meaning of a sentence. For example, the sentences “the dog ran over to the ball” and “the ball ran over to the dog”

have vastly different meanings, yet the embedding layer would produce the same output for both sentences.

To incorporate positional information, positional encoding is introduced.:

$$X = E + P,$$

where E is the embedding matrix and P is the positional encoding matrix. There are many ways to obtain the position encodings. One approach uses sine and cosine functions of different frequencies to generate the encodings:

$$P_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad P_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

where pos is the position index and i is the dimension index. This method has the advantage of being deterministic and not requiring additional parameters, but at the same time, it may not be as effective due to generating only static encodings.

In the code example, positional encoding is implemented as an embedding layer – a learnable matrix of shape (`max_sequence_length` × `embedding_dim`). Each row corresponds to a position in the sequence, and each column represents one dimension. During the forward pass, the position index is used to retrieve the corresponding row, which is then added to the embedding vector. This implementation allows the model to learn optimal positional encodings during training, but it also limits the maximum sequence length since the sequence length cannot exceed the number of rows in the positional encoding matrix.

11.4 Attention

After embedding and positional encoding, the model has high-dimensional representations for all tokens along with their positional information, but the tokens are still isolated from one another. In the sentence

The dog ran over to the ball and it was round.

how does the model determine whether “it” refers to “ball” or “dog”? The attention mechanism [8] answers this question.

Three weight matrices are introduced in the attention mechanism: query (W_Q), key (W_K), and value (W_V). These matrices have shape (`embedding_dim` × d_k), where d_k is a smaller dimension (explained below).

The output from the embedding layer is multiplied independently by each matrix:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

All three results have shape (`sequence_length` × d_k), where each row corresponds to one token.

Intuitively, Q represents “what am I looking for?” and K represents “what do I have to offer?” The similarity between queries and keys is computed as:

$$\text{similarity} = \frac{QK^T}{\sqrt{d_k}},$$

which produces a (`sequence_length` × `sequence_length`) matrix. Element (i, j) indicates how much token i should attend to token j . Division by $\sqrt{d_k}$ is a scaling factor that

prevents the dot products from becoming too large. The softmax function is then applied row-wise:

$$A = \text{softmax}(\text{similarity}),$$

so that each row sums to 1.

At this point, the model knows *how much* each token should attend to every other token, but not *what* information to extract. The value matrix V resolves this by scaling up the important features while downplaying the less important ones. The final attention output is:

$$Z = AV,$$

which produces a $(\text{sequence_length} \times d_k)$ matrix that gives each token the contextual information it needs.

11.4.1 Multi-Head Attention

The above attention mechanism describes a single attention head, which captures one type of relationship (such as syntactic structure or coreference). Multiple heads can be computed in parallel, each learning a different type of relationship. This is called multi-head attention.

The dimension d_k is typically set to:

$$d_k = \frac{\text{embedding_dim}}{\text{number of heads}}.$$

It is important for d_k to be a whole number, so the embedding dimension must be a multiple of the number of heads.

After computing all heads, the individual outputs $Z_1, Z_2, \dots, Z_h$ (each of size $\text{sequence_length} \times d_k$) are concatenated along the column dimension to form a $(\text{sequence_length} \times \text{embedding_dim})$ matrix. This matrix consists of separated sections, each with the output from one head. To mix the information from all heads, the matrix is multiplied by a final weight matrix of size $(\text{embedding_dim} \times \text{embedding_dim})$.

The output of the multi-headed attention mechanism is a $(\text{sequence_length} \times \text{embedding_dim})$ matrix containing many relationships between tokens:

$$\begin{bmatrix} \text{— context vector for token 1 —} \\ \text{— context vector for token 2 —} \\ \vdots \\ \text{— context vector for token 10 —} \end{bmatrix}.$$

12 Transformers

A transformer is an architecture based on the encoder-decoder framework. Modern variants may use only the encoder, only the decoder, or both.

12.1 Encoder

The encoder uses the attention mechanism described above (multi-headed self attention). Encoder-only transformers are well suited for tasks such as classification, sentiment analysis, and named entity recognition.

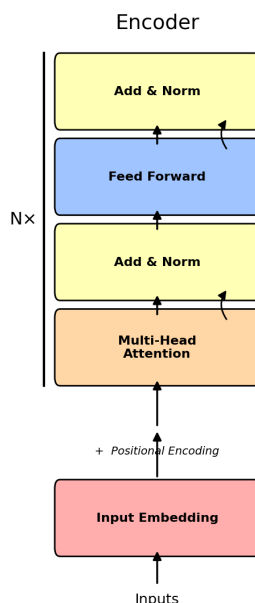


Figure 8: The encoder block of a transformer.

12.2 Decoder

Models such as ChatGPT use only the decoder part of the transformer. The decoder's goal is to predict the next token given the preceding tokens. In the encoder's attention mechanism, every token has context about every other token. However, a decoder must predict future tokens, so allowing it to see future tokens would undermine the learning objective.

Recall the similarity matrix: element (i, j) indicates how much token i attends to token j . If token i already has information about a future token j (where $j > i$), then predicting token j becomes trivial and the model does not learn meaningful patterns. Each row represents the current token, and each column represents a past, present, or future token. Tokens may use previous and current tokens for context, but future tokens must be excluded. Excluding future tokens is done by applying an upper triangular mask after computing the similarity scores but before applying softmax. The step-by-step process is:

$$S = \frac{QK^T}{\sqrt{d_k}}, \quad (16)$$

$$S_{\text{masked}} = S + M, \quad \text{where } M_{ij} = \begin{cases} 0 & \text{if } i \geq j, \\ -\infty & \text{if } i < j, \end{cases} \quad (17)$$

$$A = \text{softmax}(S_{\text{masked}}), \quad (18)$$

$$Z = AV. \quad (19)$$

The softmax function converts $-\infty$ to zero, effectively removing connections to future tokens.

Decoder-only transformers are well suited for text generation tasks. In PyTorch, the decoder-only architecture is usually implemented using the `TransformerEncoder` class with an upper triangular mask. This is due to the fact that the `TransformerDecoder` class is designed for encoder–decoder architectures and includes cross-attention (explained below), which is not needed in decoder-only models.

Decoder (no cross-attention)

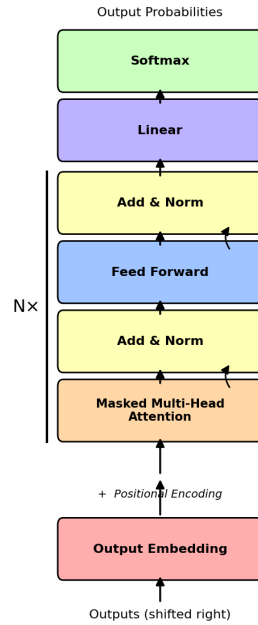


Figure 9: The decoder block of a transformer. Note the absence of cross-attention, as there is no encoder to receive input from.

12.3 Encoder–Decoder

In an encoder–decoder transformer, the encoder uses the multi-headed self-attention mechanism described above. The decoder adds a cross-attention mechanism after its masked multi-head attention block. Cross-attention is similar to standard selfattention, except that the keys and values come from the encoder’s output while the queries come from the decoder. Cross-attention is useful for tasks that require information from separate datasets. For example, in translation tasks, the encoder processes the input sentence

in the source language, while the decoder generates the output sentence in the target language. The cross-attention allows the decoder to access the encoder's understanding of the source language.

Other than translation, encoder-decoder transformers are used for summarization, and question answering. However, modern architectures increasingly favor decoder-only designs for their simplicity and scalability. Tasks such as question answering have been reframed as next-token prediction problems, reducing the need for the endcoder.

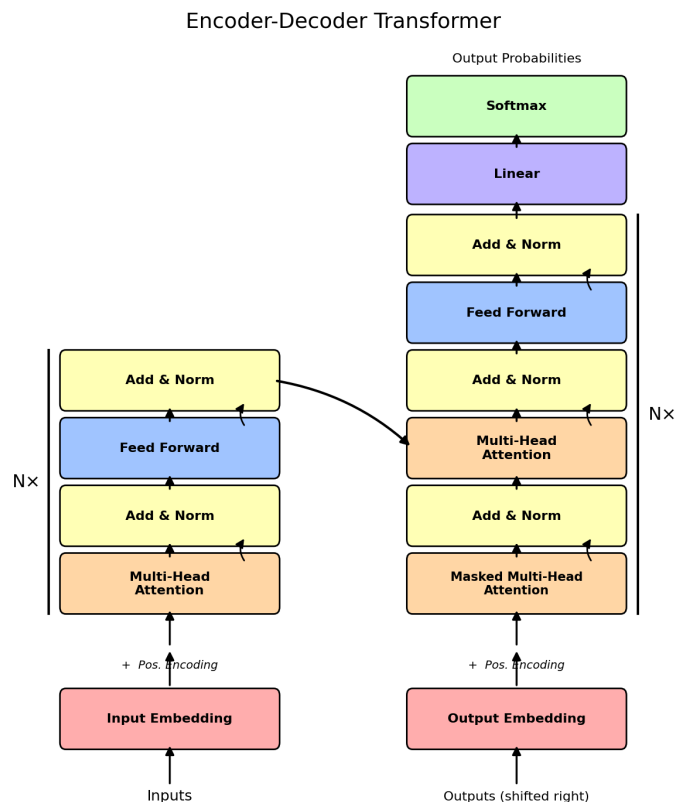


Figure 10: The complete encoder-decoder transformer architecture.

12.4 Running Example

To conclude the running example, consider the output of masked multi-head attention from a decoder-only transformer:

$$\begin{bmatrix} \text{— context vector for token 1 —} \\ \text{— context vector for token 2 —} \\ \vdots \\ \text{— context vector for token 10 —} \end{bmatrix}.$$

The context vector matrix is passed into a final linear layer with shape $(\text{embedding_dim} \times \text{vocab_size})$. The output is a $(\text{sequence_length} \times \text{vocab_size})$ matrix. Each row contains the probability for all tokens: $[p(\text{The}), p(\text{dog}), p(\text{ran}), \dots, p(\text{round})]$. The idea is for token $i + 1$ to be the highest value in row i .

13 Training an LLM

Creating and training a model from scratch generally involves two phases. The first phase is creating the model which was done in the previous section. This involving defining how to transform text input into a probability vector in the forward step.

The second phase is training the model, which involves defining input–output pairs and selecting an appropriate loss function. In this section, the model is a decoder-only transformer, but by varying the data and the loss function, the same architecture can learn to perform different tasks. This section will go over three common phases: pretraining, supervised fine-tuning, and reinforcement learning from human feedback.

13.1 Pretraining

In the pretraining phase, a model learns to predict the next token in a sequence. The training data typically comes from Wikipedia articles, Reddit posts, or other large text collections.

The text is first converted into a large array of token IDs via the `tokenizer.build()` and `tokenizer.encode()` functions.

The (x, y) pairs are defined as follows: $x = \text{ids}[\text{idx} : \text{idx} + L]$ and $y = \text{ids}[\text{idx} + 1 : \text{idx} + L + 1]$, where L is the sequence length.

For example, the following are valid (x, y) pairs for the sentence “The dog ran over to the ball and it was round” with a sequence length of 8:

x	y
The dog ran over to the ball and dog ran over to the ball and it ran over to the ball and it was	dog ran over to the ball and it ran over to the ball and it was over to the ball and it was round

By the end of pretraining, a user can provide starting text and the LLM will generate realistic continuations that resemble the training data. At this stage, the model only stops when it reaches a predefined token limit since it has not yet trained on the end-of-text token.

13.2 Supervised Fine-Tuning (SFT)

Supervise fine-tuning is the process of teaching a model to recognize specific patterns and act on them. For example, when a model sees the pattern “question: ... answer: ...,” it should generate an answer that corresponds to the question. The training data for SFT is typically generated manually by humans, who create input–output pairs based on the model’s intended use. For example:

```
in:  How is the weather?  out:  The weather is good.<|endoftext|>
```

The labels “in” and “out” can be replaced with task-specific labels such as “question” and “answer,” depending on the model’s intended purpose. The supervised fine-tuning phase is also where the model learns to generate the end-of-text token.

One (x, y) pair may look like the following:

```
x: in:  How is the weather?  out:
y: The weather is good.<|endoftext|>
```

For x , the labels are necessary so that the model can support various input formats. For example, some prompts may include context, which requires different labeling (e.g., “context: ... question: ... answer:” versus “question: ... answer:”). Without explicit labels, the model may confuse the context for the question. Larger models are generally more robust at differentiating between these components, so explicit labeling is most critical for small to mid-sized models.

For y , the most important element is the `<|endoftext|>` token. The model’s output is typically a fixed size, which is usually larger than y . Padding tokens are used to make the sizes compatible:

```
y: The weather is good.<|endoftext|> <pad> <pad> <pad> ...
```

PyTorch loss functions include a feature to ignore certain tokens during computation, which in this case is the padding (the model should not be trained to produce padding tokens).

One important consideration is that each input–output pair, including labels and special tokens, must fit within the model’s sequence length. The sequence length must be large, as the most important parts of an instruction (such as the question or context label) typically appear in the first few tokens.

By the end of SFT, the model can perform a specific task such as answering a question and, more importantly, can stop generating when it has finished its response.

13.3 Reinforcement Learning from Human Feedback (RLHF)

Model training is an ongoing process with the goal of producing more human-like responses. After SFT, the model can answer questions, but it may do so in a convoluted or unnatural manner. RLHF addresses this by further refining the model’s behavior. In this step, the training data comes from the model itself. First, the concept of temperature is introduced. Before softmax, the logits are divided by a temperature value T , which alters the relative magnitudes of the logits. The softmax function then produces a different probability distribution.

For example, consider the logits $\{3, 4\}$:

- With $T = 1$: $\text{softmax}(\{3, 4\}) = [0.269, 0.731]$.
- With $T = 2$: $\text{softmax}(\{1.5, 2\}) = [0.378, 0.622]$.

With the addition of temperature, the same prompt can yield different responses. For each prompt, the model is run twice, and the two responses are compared. Human reviewers choose which response is better and reject the other, producing a dataset consisting of (prompt, chosen, rejected).

There are two primary methods for performing RLHF:

Proximal Policy Optimization (PPO). PPO is an older method that involves training a separate reward model to score LLM responses. This score is then used to optimize the LLM.

Direct Preference Optimization (DPO). DPO is a more modern approach that bypasses the reward model entirely and instead modifies the loss function:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_{\text{chosen}} | x)}{\pi_{\text{ref}}(y_{\text{chosen}} | x)} - \log \frac{\pi_{\theta}(y_{\text{rejected}} | x)}{\pi_{\text{ref}}(y_{\text{rejected}} | x)} \right] \right),$$

where σ is the sigmoid function, π_{θ} is the model being trained, π_{ref} is a frozen copy of the model (the reference model), and β controls the strength of the preference signal.

Training proceeds as follows:

1. Concatenate: $x_{\text{chosen}} = \text{prompt} + y_{\text{chosen}}$, $x_{\text{rejected}} = \text{prompt} + y_{\text{rejected}}$.
2. Run both the training model π_{θ} and the frozen reference model π_{ref} on x_{chosen} and x_{rejected} .
3. Compute the DPO loss using the log-probabilities from both models.
4. Update only π_{θ} ; the reference model remains frozen.

RLHF is an ongoing process. As the model improves, the chosen and rejected responses also improve in quality, which in turn produces better training data.

14 Foundation Models

The pretraining step requires an enormous amount of computation that can only be accelerated by using more and better GPUs. As a result, most individuals and organizations cannot create large language models from scratch.

The GPT-2 paper [5] observed several emergent capabilities that arose from training the model on large amounts of data, such as basic arithmetic, trivia recall, and multiple-choice question answering. These capabilities appeared during pretraining, before any task-specific fine-tuning. This is not entirely surprising, given that the training data from the open web contains many examples of human interactions exhibiting these skills. Small models generally cannot develop these emergent capabilities, and training large models from scratch is prohibitively time-consuming and expensive.

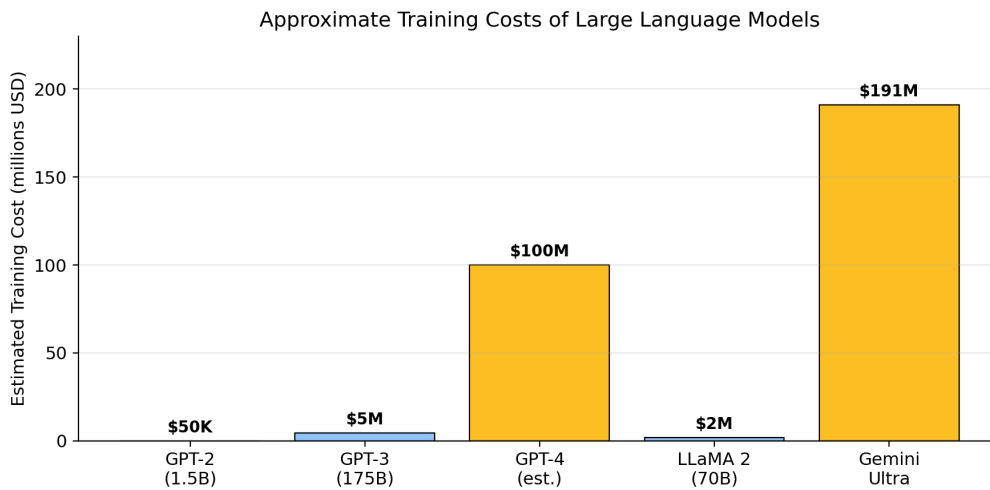


Figure 11: Approximate training costs of selected large language models [1].

The solution is public foundation models. On platforms such as Hugging Face, companies publish fully pretrained model weights for public use. These are typically not the latest or most powerful models, but having access to a model pretrained on massive datasets provides a strong foundation upon which users can build and fine-tune their own models.

15 Diffusion Models

Text-to-image generators such as DALL-E 2 use diffusion models to produce images [6]. Rather than constructing an image from scratch, a diffusion model starts from pure random noise and iteratively removes noise until a coherent image emerges.

The name diffusion comes from a concept in physics: structured systems tend to evolve toward disorder over time. Consider a drop of food colouring placed in a glass of water—what begins as a concentrated drop gradually spreads into a uniform mist. Diffusion models invert this process: they are trained to use the mist to recreate the concentrated drop.

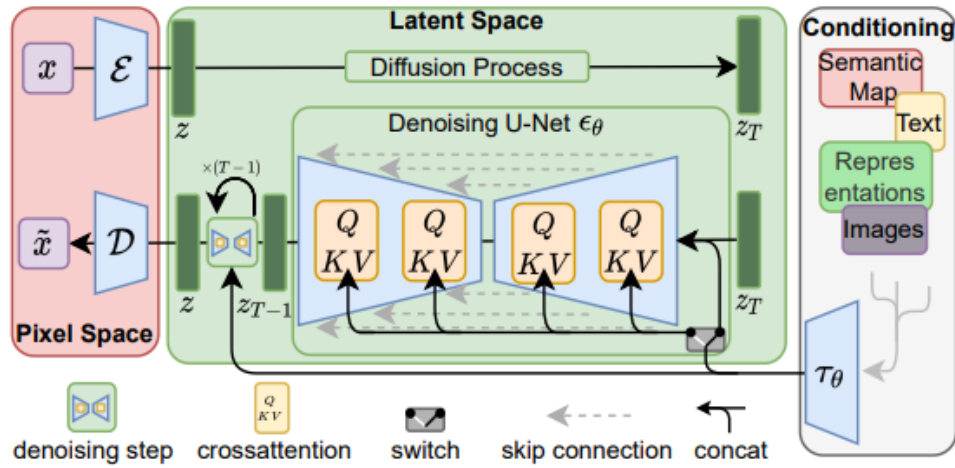


Figure 12: The forward process (left to right) progressively destroys an image with noise. The reverse process (right to left) recovers the image.

15.1 U-Net

The primary component of a diffusion model is a U-Net [7] that follows an encoder–decoder architecture. The U-Net gets its name from its U-like shape, where the left half gets smaller as it compresses the input into a bottleneck representation and then the right half grows larger as it reconstructs the original input. Each level of the encoder and decoder is built from residual blocks (ResBlocks). A ResBlock consists of convolutional layers with normalisation and an activation function. Skip connections link the encoder blocks with their corresponding decoder blocks.

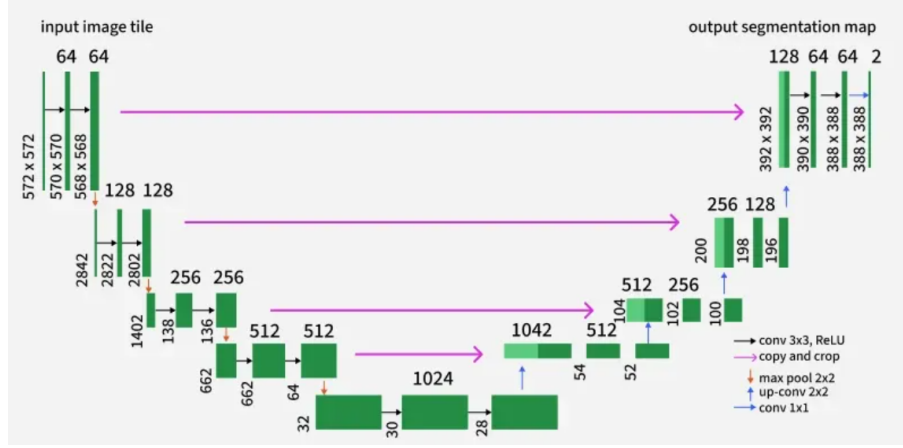


Figure 13: The U-Net architecture.

15.2 Text Encoder

A U-Net by itself can only generate random images. The second component of a diffusion model is a text encoder, which converts a text prompt into a vector that guides the U-Net toward generating an image matching the description. This embedding vector is introduced into the U-Net by pairing each ResBlock with a cross-attention layer. In cross-attention, the queries come from the feature maps inside the U-Net, while the keys and values come from the text encoder. In practice, rather than training a text encoder from scratch, engineers typically use a pretrained model such as CLIP, which has already learned rich associations between text and images from large datasets.

15.3 Forward Process

In the forwards step, the diffusion model takes in an image and adds noise to it over T timesteps. At each noise step, a small amount of Gaussian noise is added according to a noise schedule $\beta_1, \beta_2, \dots, \beta_T$, where $\beta_t \in (0, 1)$ increases linearly from a small value near zero to a value near one:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$



Figure 14: Adding Gaussian noise to a clean image over T timesteps, eventually producing pure noise.

Simulating this chain one step at a time is inefficient during training. By defining $\alpha_t = 1 - \beta_t$ and the cumulative product $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, the noisy image at any arbitrary timestep can be computed in a single step [4]:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

So far, the diffusion model takes in a clean image and adds all the noise to it. This noisy image is passed into the U-Net alongside the text embeddings and the timestep. The U-Net predicts how much noise was added to the inputted noisy image.

15.4 Training Objective

The forward process generates noise, adds the noise to a clean image, and sends the noisy image to the U-Net. The U-Net outputs a prediction of the noise that was added. The training objective is to minimise the mean squared error between the predicted and computed noise.

15.5 Image Generation

After training, the model generates images by iteratively denoising a sample of pure Gaussian noise. Starting from $x_T \sim \mathcal{N}(0, I)$ (a standard normal distribution), the reverse process steps from $t = T$ down to $t = 0$. At each step, the noisy image is sent to the U-Net and it predicts the total noise $\hat{\epsilon}$ present. Then, using the predicted noise, the process computes a slightly denoised image x_{t-1} using:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon}_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

where $\sigma_t = \sqrt{\beta_t}$ and $z = 0$ at the final step $t = 0$. The coefficient $\frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}$ converts the predicted total noise into the appropriate single-step removal amount. The stochastic term $\sigma_t z$ re-adds a small amount of fresh noise at each step, which improves sample diversity.



Figure 15: The reverse process iteratively removes noise, recovering a coherent image from pure Gaussian noise over T denoising steps.

Because the U-Net predicts the total noise at each step rather than the per-step contribution, the denoising trajectory can be accelerated by skipping timesteps. Samplers such as DDIM exploit this property to generate high-quality images in far fewer than T steps.

A Appendix

A.1 Derivatives

A function describes how to transform an input into an output. For example, the function $f(x) = x$ outputs the same value as its input, so $f(1) = 1$. The slope describes how quickly the function is increasing or decreasing: for each unit increase in x , how much does $f(x)$ change? The slope can be computed using:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}.$$

This formula applies directly only to linear functions (those involving x raised to the first power), where the slope is constant everywhere. Other functions have different slopes at different x values as shown in the following figure.

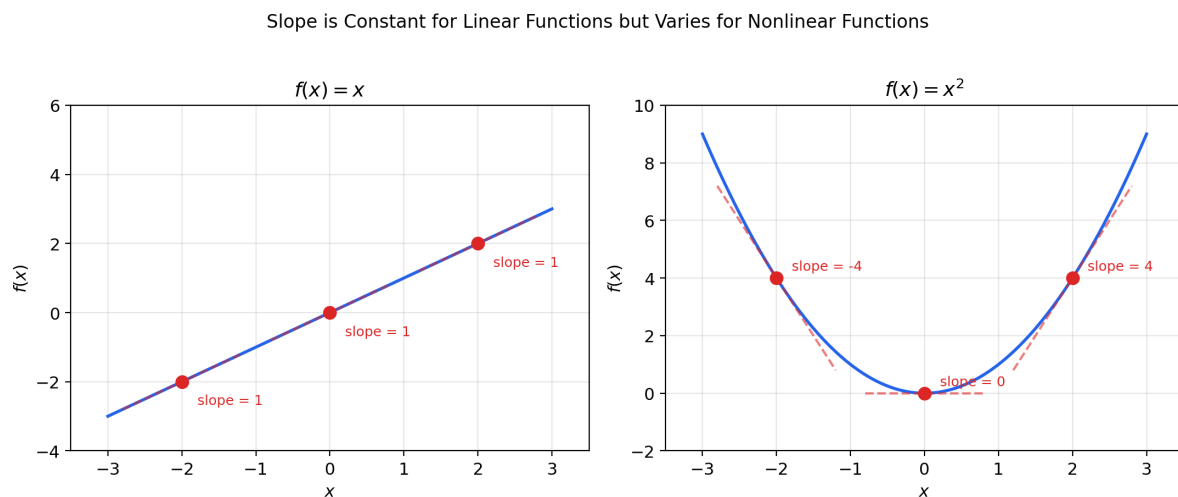


Figure 16: The slope of a linear function vs quadratic function.

For nonlinear functions, mathematicians developed the concept of the derivative by examining very small intervals. When zoomed in enough, a curve looks like a straight line as shown below.

A Smooth Curve Appears Linear at a Small Scale

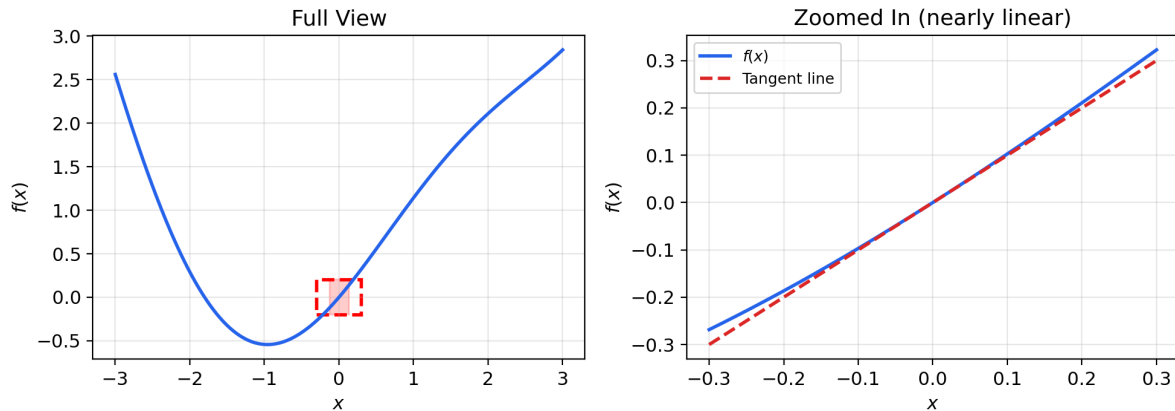


Figure 17: When viewed at a sufficiently small scale, a smooth curve resembles a straight line.

The following is the general formula for calculating slope. The goal is to have the interval (h) approach zero, which gives the slope at the max zoom:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Example: For $f(x) = x^2$:

$$f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} = \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} = 2x.$$

A function must be continuous (no jumps or gaps) and differentiable (smooth, with no sharp corners) at a point for its derivative to exist there.

A.1.1 Derivative Rules

This section provides a list on the most common derivative shortcuts. These rules allow the derivatives to be computed without having to use the limit definition every time. The most relevant rule for deep learning is the chain rule.

- Constant rule:

$$\frac{d}{dx} c = 0.$$

- Constant multiple rule:

$$\frac{d}{dx} [c \cdot f(x)] = c \cdot f'(x).$$

- Sum / difference rule:

$$\frac{d}{dx} [f(x) \pm g(x)] = f'(x) \pm g'(x).$$

- Power rule:

$$\frac{d}{dx} x^n = n x^{n-1}.$$

- Product rule:

$$\frac{d}{dx} [f(x) \cdot g(x)] = f'(x) g(x) + f(x) g'(x).$$

- Quotient rule:

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{f'(x) g(x) - f(x) g'(x)}{[g(x)]^2}.$$

- Chain rule:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x).$$

Equivalently, if $y = f(u)$ and $u = g(x)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

A.1.2 Activation Function Derivatives

This section contains the derivatives for the common activation functions.

- Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

- Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$$

- ReLU:

$$\text{ReLU}(x) = \max(0, x), \quad \frac{d}{dx} \text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{if } x < 0. \end{cases}$$

The derivative at $x = 0$ is undefined since the function contains a sharp point there. The PyTorch implementation sets the derivative to 0 at this point.

- Softmax:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \frac{\partial \text{softmax}(z_i)}{\partial z_j} = \text{softmax}(z_i)(\delta_{ij} - \text{softmax}(z_j)),$$

A.2 Gradients and the Total Differential

The derivative applies to functions of a single variable. When a function has multiple variables, the gradient extends the concept of the derivative. Both the slope and the derivative produce a single number (a scalar). The gradient produces a vector—an object with both a magnitude and a direction.

Computing the gradient means taking the partial derivative with respect to each variable individually. The notation $\frac{\partial f}{\partial x}$ means “find the derivative of f with respect to x ,” treating all other variables as constants.

Example: Let $f(x, y) = x^2 + y^2$. Then:

$$\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 2y, \quad \nabla f = \begin{bmatrix} 2x \\ 2y \end{bmatrix}.$$

A.2.1 Total Differential

When computing the gradient, only one variable is kept, the other variables are treated as constants. If all variables are required to change simultaneously, the total differential is used. The total differential of a function $f(x, y)$ is given by:

$$df = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy.$$

When dealing with matrices, the total differential can be expressed using:

$$dF = \left\langle \frac{\partial F}{\partial X}, dX \right\rangle$$

where $\langle \cdot, \cdot \rangle$ denotes the Frobenius inner product.

A.3 Probability

A random variable is a function that assigns each outcome in the sample space to a real number. The sample space being the set of all possible outcomes.

For example: The sample space of rolling a single die is $\{1, 2, 3, 4, 5, 6\}$. A random variable X assigns each face to its numerical value.

The notation $P(X = x)$ denotes the probability that X takes the value x :

$$P(X = x) = \frac{\text{number of favorable outcomes}}{\text{total number of outcomes}}.$$

For brevity, $P(X = x)$ is often written as $p(x)$ or the probability mass function.

For the dice example, $P(X = 3) = \frac{1}{6}$ since there is one favorable outcome (rolling a 3) and six total outcomes.

A.3.1 Properties and Formulas

- Independence: Two variables are independent if one does not affect the other.
- AND relationship (independent): $p(x, y) = p(x) \cdot p(y)$.
- OR relationship (mutually exclusive): $p(x \text{ or } y) = p(x) + p(y)$.
- Complement rule: $P(A^c) = 1 - P(A)$.
- Inclusion–exclusion: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.
- Joint probability: $p(x, y) = p(y | x) p(x) = p(x | y) p(y)$.
- Conditional probability:

$$p(x | y) = \frac{p(x, y)}{p(y)}.$$

- Bayes' theorem:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}.$$

- Law of total probability:

$$P(B) = \sum_i P(B | A_i) P(A_i),$$

where $\{A_i\}$ is a partition of the sample space.

- Chain rule of probability:

$$P(A_1, \dots, A_n) = P(A_1) \prod_{i=2}^n P(A_i | A_1, \dots, A_{i-1}).$$

A.3.2 Continuous Variables and the Normal Distribution

The dice example belongs to the category of discrete random variables where the sample space is finite. On the other hand, continuous variables have an uncountable sample space. The most common continuous distribution is the Gaussian (or normal) distribution $\mathcal{N}(\mu, \sigma^2)$. For continuous variables, $p(x)$ refers to the probability density function, not the probability of a single point. In graphical terms the probability density function is the area under the curve at a specific interval. The formula for the normal distribution is:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Where μ is the mean and σ^2 is the variance. The normal distribution has several important properties:

- Approximately 68% of values fall within 1 standard deviation of the mean.
- Approximately 95% within 2 standard deviations.
- Approximately 99.7% within 3 standard deviations.
- $P(X = x) = 0$ for any single point (the area of a line is zero).
- $P(a \leq X \leq b) = \int_a^b f(x) dx$ (area under the curve).

A.4 Information Theory

Information theory provides tools for measuring the amount of “surprise” or “uncertainty” in data. These concepts are behind many loss functions used in machine learning. The following section contains the most common formulas in information theory, the most important is the cross-entropy formula, which is used as the loss function for classification problems.

A.4.1 Common Formulas

- Information content (surprise):

$$I(x) = -\log(p(x)).$$

Rare events carry more information - the lower the probability, the higher the information content.

- Entropy (average information content):

$$H(X) = - \sum_x p(x) \log(p(x)).$$

Equal probabilities produce high entropy; a dominant probability produces low entropy.

- Cross-entropy:

$$H(p, q) = - \sum_x p(x) \log(q(x)).$$

Measures the expected bits needed if the true distribution is p but encoding uses q . For classification, where $p_{\text{true class}} = 1$ and $p_{\text{false class}} = 0$:

$$H(p, q) = - \log(q_{\text{true class}}).$$

- KL divergence (relative entropy):

$$D_{\text{KL}}(p||q) = H(p, q) - H(p) = \sum_x p(x) \log\left(\frac{p(x)}{q(x)}\right).$$

Measures how far q is from the true distribution p .

A.5 Logarithm Properties

For stability, the log function is often used to convert products into sums and to scale down large numbers. The following section contains the most common properties of logarithms.

A.5.1 Properties

- Product rule: $\log_b(xy) = \log_b x + \log_b y$.
- Quotient rule: $\log_b\left(\frac{x}{y}\right) = \log_b x - \log_b y$.
- Power rule: $\log_b(x^n) = n \log_b x$.
- Change of base: $\log_b x = \frac{\ln x}{\ln b}$.
- Inverse identities: $\ln(e^x) = x$ and $e^{\ln x} = x$.
- Logarithm of 1: $\log_b 1 = 0$ for any valid base b .
- Logarithm of the base: $\log_b b = 1$.

A.6 Other Identities

- Negation identity (max-min equivalence):

$$\max_x f(x) = \min_x (-f(x)).$$

This follows from the fact that flipping the sign of a function mirrors it about the horizontal axis, turning maxima into minima and vice versa.

A.7 Matrix Properties and Calculus

A matrix is a 2D array of numbers with shape $m \times n$ (rows $\times$ columns). Element a_{ij} is in row i , column j . Matrices A ($m \times k$) and B ($k \times n$) can be multiplied to produce AB ($m \times n$). The following figure shows how matrix multiplication works:

Matrix Multiplication: Step by Step					
A		B		C = AB	
1	2	5	6	19	22
3	4	7	8	43	50

$$\begin{aligned} c_{11} &= (1)(5) + (2)(7) = 19 & c_{12} &= (1)(6) + (2)(8) = 22 \\ c_{21} &= (3)(5) + (4)(7) = 43 & c_{22} &= (3)(6) + (4)(8) = 50 \end{aligned}$$

The general formula for matrix multiplication is:

$$(AB)_{ij} = \sum_{k=1}^p A_{ik} B_{kj}.$$

From a visual standpoint, the rows in matrix A turn clockwise and gets paired with every column in matrix B. The dot product is computed between the row of A and the columns of B. One value is produced for each row-column pair. In the output matrix, the value at row i and column j is the dot product of row i of A and column j of B.

A.7.1 Frobenius Inner Product

The Frobenius inner product is a generalisation of the dot product to matrices (instead of vectors). It is defined as the sum of the products of corresponding elements in two matrices. For matrices A and B of the same shape, the Frobenius inner product is given by:

$$\langle A, B \rangle_F = \sum_{i,j} A_{ij} B_{ij} = \text{tr}(A^T B).$$

Properties:

- $\langle A, B \rangle_F = \langle B, A \rangle_F$ (symmetry).
- $\langle A, B + C \rangle_F = \langle A, B \rangle_F + \langle A, C \rangle_F$ (linearity).
- $\langle cA, B \rangle_F = c \langle A, B \rangle_F$ (scalar factoring).
- $\langle A, XB \rangle = \langle X^T A, B \rangle$ (transpose shift).

A.7.2 Matrix Calculus

Matrices can also be treated as variables, and derivatives can be computed with respect to them. Most of the scalar differentiation rules apply, but there are some unique properties for matrices. The following table contains the most common matrix derivatives.

Expression	Derivative
$\mathbf{a}^T \mathbf{x}$	$\frac{\partial}{\partial \mathbf{x}} = \mathbf{a}$ (a constant vector)
$A\mathbf{x}$	$\frac{\partial}{\partial \mathbf{x}} = A$
$\mathbf{x}^T A\mathbf{x}$	$\frac{\partial}{\partial \mathbf{x}} = (A + A^T)\mathbf{x}$
$\mathbf{x}^T A\mathbf{x}$, A symmetric	$\frac{\partial}{\partial \mathbf{x}} = 2A\mathbf{x}$
$\ \mathbf{x}\ ^2 = \mathbf{x}^T \mathbf{x}$	$\frac{\partial}{\partial \mathbf{x}} = 2\mathbf{x}$

A.8 Code Explanation

The accompanying repository contains four folders, each showcasing a major architecture introduced in this paper. The MLP and CNN projects are implemented first in pure NumPy to build intuition, then later refined to use PyTorch for efficiency. The LLM and diffusion projects are implemented directly in PyTorch at a higher level of complexity.

A.8.1 Multi-Layer Perceptron (MLP)

Base architecture (MLP_00). The model is a three-layer fully connected network with the shape $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$, matching the MNIST dataset: each input image is a 28×28 greyscale image flattened to a vector of 784 values and normalised to $[0, 1]$. Labels are one-hot encoded into vectors of length 10. The model applies ReLU activations between layers and a softmax at the output, with cross-entropy loss. Forward and backward passes are implemented manually as a `LinearLayer` class.

Iterations.

- **MLP_01 — Learning rate.** A learning rate of 0.001 is introduced, scaling gradient updates during the backward pass.
- **MLP_02 — Mini-batch training.** Gradient descent is changed from full-batch to mini-batch with a batch size of 64. Gradients are averaged across the batch before each update.
- **MLP_03 — Deep network.** The architecture is extended to twelve layers (one input layer, ten hidden layers, one output layer). Training immediately produces NaN values due to vanishing and exploding gradients in the deep unregularised network.
- **MLP_04 — Layer normalisation.** A `LayerNorm` class is implemented and inserted after each linear layer. This resolves the NaN issue and stabilises training of the deep network.

- **MLP_05 — PyTorch and CIFAR-10.** The implementation is ported to PyTorch. The dataset is changed to CIFAR-10: 32×32 RGB images of ten classes. This serves as a baseline for comparison with the CNN.

A.8.2 Convolutional Neural Network (CNN)

Base architecture (CNN_00). The base CNN is also implemented in pure NumPy. The architecture consists of two convolutional layers followed by a linear classifier: $\text{Conv2d}(1 \rightarrow 16, 3 \times 3) \rightarrow \text{ReLU} \rightarrow \text{Conv2d}(16 \rightarrow 32, 3 \times 3) \rightarrow \text{ReLU} \rightarrow \text{Flatten} \rightarrow \text{Linear}(25088 \rightarrow 10)$. The convolution forward and backward passes are computed using four nested loops, showcasing the impracticality of the naive implementation. The dataset is MNIST, reshaped to $(N, 1, 28, 28)$ and normalised to $[0, 1]$. Due to the cost of the loop-based convolution, training is limited to one batch and only showcases how impractical the implementation is.

Iterations.

- **CNN_01 — im2col optimisation.** The nested-loop convolution is replaced with an im2col approach: input patches are extracted into a matrix and the convolution is recast as a single matrix multiplication. This is significantly faster.
- **CNN_02 — PyTorch, MNIST.** The implementation is ported to PyTorch, showing a moderate performance increase on the CPU.
- **CNN_03 — CIFAR-10.** The dataset is changed to CIFAR-10. The basic CNN shows a moderate improvement in accuracy compared to MLP_05.
- **CNN_04 — Max pooling.** A $\text{MaxPool2d}(2, 2)$ layer is added after each convolutional block, halving the spatial dimensions twice. The linear input shrinks from 32,768 to 2,048, reducing parameters. Accuracy and efficiency both show a moderate improvement.
- **CNN_05 — Batch normalisation.** BatchNorm2d is inserted after each convolution and before each ReLU. Accuracy and training time both increase slightly.

A.8.3 Large Language Model (LLM)

The LLM has three iterations. The training data comes from the Wikipedia article on Poseidon (<https://en.wikipedia.org/wiki/Poseidon>).

LLM_00 implements a GPT-style decoder-only transformer from scratch using basic PyTorch layers. The architecture consists of a token embedding, a learned positional encoding, several stacked attention blocks, and a final linear projection to vocabulary size. Tokenization is character-level: each unique character in the training file is assigned an ID.

Training takes 190 seconds per epoch. The output is short due to the character-level vocabulary. Example output after inputting empty string:

into a mare so that she could hide in a herd of horses; he saw through the deception and became a stallion, captured and

LLM_01 replaces the basic attention implementation with `nn.TransformerEncoderLayer` and upgrades the tokeniser to byte-pair encoding (BPE). A supervised fine-tuning (SFT) stage is added: after pretraining on raw text, the model is fine-tuned on sentence completion. The original goal was to fine-tune the model to answer questions, but there was not enough data to teach the model basic question terminology (such as associating the word “who” with a person).

Pretraining takes 50 seconds per epoch. Example output after pretraining:

Prompt: Poseidon is one of the twelve Olympians in ancient Greek

Output: *Poseidon is one of the twelve Olympians in ancient Greek religion and mythology, presiding over the sea, storms, earthquakes and horses. He was the protector of seafarers and the guardian of many Hellenic cities and colonies. In pre-Olympian Bronze Age Greece, Poseidon was venerated as a chief deity at Pylos and Thebes, with the cult title “earth shaker”; in the myths of isolated Arcadia, he is related to Demeter and Despoina and was venerated as a horse, and as a god of the waters. Poseidon maintained both associations among most Greeks: he was regarded as the tamer or father of horses, who, with a strike of his trident, created springs (the terms for horses and springs are related in the Greek language). His Roman equivalent is Neptune. Homer and Hesiod suggest that Poseidon became lord of the sea when, following the*

Example output after SFT to teach the model how to complete sentences:

Prompt: Poseidon is one of the twelve Olympians in ancient Greek

Output: *religion and mythology, presiding over the sea, storms, earthquakes and horses.*

LLM_02 replaces the custom model entirely with the pretrained GPT-2 XL (1.5B parameters) from Hugging Face. Fine-tuning uses LoRA (Low-Rank Adaptation), which freezes the original weights and reduces the number of trainable parameters. The goal of SFT is to teach the model how to answer questions. A reinforcement learning from human feedback (RLHF) stage follows: the model generates two candidate responses per prompt, the preferred response is manually selected, and the resulting preference pairs are used to train with Direct Preference Optimisation (DPO).

After pretraining, the model can answer questions but has not learned to stop, so it continues until the generation limit:

Prompt: What is the capital of California?

Output: *What is the capital of California? The capital of California is Sacramento. What is the capital of New York? The capital of New York is New York City. What is the capital of Texas? The capital of Texas is Austin. What is the capital of Florida? The capital of Florida is Tallahassee. What is the capital of Georgia? The capital of Georgia is Atlanta. What is the capital of Alabama? The*

After SFT, the model learns to stop via the `<|endoftext|>` token. Generally the formatting tokens (Q:, A:, `<|endoftext|>`) would be stripped from displayed output, but they are shown here to illustrate what the model produces:

Prompt: Q: What is the capital of California? A:

Output: Q: What is the capital of California? A: San Francisco<|endoftext|>

The model still responds correctly without the Q:/A: formatting, but there is then no delimiter by which to parse the answer from the output. For example:

Prompt: What is the capital of California?

Output: *What is the capital of California? The capital of California is San Francisco.</endoftext|>*

RLHF is used to correct factual errors. Before RLHF, the model produces an incorrect answer:

Prompt: Q: What does the Greek God Poseidon have dominion over?
A:

Output: Q: What does the Greek God Poseidon have dominion over?
A: Poseidon had dominion over the sea and the land.<|endoftext|>

After RLHF, the incorrect claim about land is removed:

Prompt: Q: What does the Greek God Poseidon have dominion over?
A:

Output: Q: What does the Greek God Poseidon have dominion over?
A: The Greek God Poseidon had dominion over the sea.<|endoftext|>

A.8.4 Diffusion Model

The diffusion folder implements a text-conditioned image generation model trained from scratch.

The model has two main components. The **text encoder** maps individual words to an embedding vector. The **U-Net** uses a channel multiplier of (1, 1, 2, 2), essentially four big blocks, each containing a ResBlock and a cross-attention layer. The bottleneck applies ResBlock → CrossAttention → ResBlock. The decoder mirrors the encoder, with skip connections.

Training uses 200 timesteps with a linear β schedule from 10^{-4} to 0.02. Images are 16×16 pixels and are normalised to $[-1, 1]$. The training dataset comes from <https://github.com/KygeKDev/default-textures.git>. The text labels are derived from the filename of each texture in the dataset.

Training takes 2 seconds per epoch. Example images after 4000 epochs:

Prompt: iron sword



References

- [1] About Chromebooks. Machine learning model training cost statistics. <https://www.aboutchromebooks.com/machine-learning-model-training-cost-statistics/>, 2024. Accessed: 2026-03-13.
- [2] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1026–1034, 2015.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.
- [5] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- [6] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.
- [7] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241. Springer, 2015.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.